Atelier 4 — Git et GitHub : travail collaboratif

Corrigé pratique | Binôme : Développeur 1 et Développeur 2

Document : Corrigé rédigé à partir des supports fournis

Objectif et résultat attendu

Deux développeurs collaborent sur un site simple sans travailler directement sur la branche principale. Le dépôt distant contient trois pages : index.html, contact.html et promotion.html. Le corrigé retient le nom de branche main ; si le dépôt utilise master, remplacer main partout de manière cohérente.

Partie 1 — Dépôt distant et première version (Développeur 1)

1. Créer sur GitHub un dépôt vide nommé atelier-collaboratif, puis copier son URL SSH. Cloner le dépôt et se placer dedans :

```
git clone git@github.com:VOTRE-UTILISATEUR/atelier-collaboratif.git
cd atelier-collaboratif
git status
git branch
git remote -v
```

Créer la page d'accueil index.html :

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Atelier collaboratif</title>
</head>
<body>
  <h1>Bienvenue sur notre site</h1>
  <p>Projet réalisé en équipe avec Git et GitHub.</p>
</body>
</html>
```

```
git status
git add index.html
git status
git commit -m "Ajouter la page d'accueil"
git branch -M main
git push --set-upstream origin main
```

Réponses aux premières questions

- **git clone :** copie le dépôt distant, son historique et ses références de branches dans un répertoire local.
- **Dossier créé :** atelier-collaboratif, qui contient les fichiers du projet et le répertoire caché .git.
- **origin :** nom conventionnel du dépôt distant associé au dépôt local ; ce n'est ni une commande ni une branche.
- **Dépôt local / distant :** le dépôt local se trouve sur la machine du développeur ; le dépôt distant est hébergé sur GitHub et sert au partage/synchronisation du projet.

Partie 2 — Branche Contact (Développeur 1)

Créer la branche dédiée et la page de contact, puis publier la branche :

```
git switch -c contact
```

```
<!doctype html>
<html lang="fr">
<head><meta charset="utf-8"><title>Contact</title></head>
<body>
  <h1>Contactez-nous</h1>
  <p>Email : contact@exemple.com</p>
</body>
</html>
```

```
git add contact.html
git commit -m "Ajouter la page contact"
git push --set-upstream origin contact
```

La branche contact est maintenant publiée sur GitHub. Le premier push avec --set-upstream configure la branche distante suivie ; les prochains git push et git pull sont ainsi plus simples.

Partie 3 — Ajouter le collaborateur

Dans les paramètres du dépôt GitHub, inviter le compte du Développeur 2 en tant que collaborateur. Le Développeur 2 accepte l'invitation ; les droits effectifs sont ceux accordés par le propriétaire du dépôt.

Partie 4 — Branche Promotion (Développeur 2)

Le deuxième développeur clone le dépôt, part de main et crée une branche indépendante :

```
git clone git@github.com:UTILISATEUR-DEV1/atelier-collaboratif.git
cd atelier-collaboratif
git branch
git branch -r
git remote -v
git switch main
git pull origin main
git switch -c promotion
```

Créer promotion.html avec le contenu suivant, puis commiter et publier :

```
<!doctype html>
<html lang="fr">
<head><meta charset="utf-8"><title>Promotion</title></head>
<body>
  <h1>Nos promotions</h1>
  <p>Découvrez nos offres du moment.</p>
</body>
</html>
```

```
git add promotion.html
git commit -m "Ajouter la page promotion"
git push --set-upstream origin promotion
```

Partie 5 — Pull Requests et intégration

Sur GitHub, le Développeur 1 ouvre une Pull Request de contact vers main, donne le titre « Ajouter la page Contact », relit le diff puis fusionne après validation. Le Développeur 2 répète la même opération de promotion vers main avec le titre « Ajouter la page Promotion ».

Après les fusions, chaque développeur synchronise sa branche principale locale :

```
git switch main
git pull origin main
```

Questions de synthèse — réponses

1. Différence entre dépôt local et distant ?
Le dépôt local conserve les fichiers, commits et branches sur l'ordinateur ; le dépôt distant, ici GitHub, héberge la version partagée et reçoit les pushes.

2. Pourquoi travailler sur des branches distinctes ?
Pour isoler les changements, limiter les conflits sur la branche principale, faciliter la revue et permettre plusieurs développements en parallèle.

3. Rôle de origin dans git push origin contact ?
origin désigne le dépôt distant configuré ; contact est la branche envoyée.

4. Pourquoi --set-upstream au premier push ?
Pour relier la branche locale à la branche distante correspondante, et simplifier les prochains push/pull.

5. Différence entre git push et Pull Request ?
git push publie des commits sur un dépôt distant. Une Pull Request est une demande de revue et d'intégration entre branches sur GitHub.

6. Une Pull Request fusionne-t-elle automatiquement la branche ?
Non. Elle présente les changements et permet leur revue ; la fusion intervient lorsqu'une personne autorisée l'approuve et déclenche le merge.

7. Pourquoi récupérer main après une fusion ?
Pour intégrer localement les modifications ajoutées sur GitHub et éviter de travailler sur une version dépassée.

8. Combien de branches créées et quelles fonctions ?
Deux branches de fonctionnalité sont créées explicitement : contact pour la page de contact et promotion pour la page de promotion. Elles partent de main, la branche principale préexistante.

Questions complémentaires — commandes Git

Commande	Rôle
git add fichier	Prépare les modifications pour un commit.
git commit -m "message"	Enregistre les changements préparés dans l'historique local.
git push origin branche	Publie les commits de la branche vers le dépôt distant.
git pull origin main	Récupère et intègre les mises à jour de main depuis origin.
git switch -c nom	Crée une branche puis bascule dessus.

État final attendu : la branche main contient index.html, contact.html et promotion.html, ajoutés au moyen de deux branches distinctes puis de Pull Requests.